

Unit 3

JSP

❖ Introduction to JSP and JSP Basics

What is JSP?

- **Java Server Pages (JSP)** is a **server-side technology** used to create **dynamic, platform-independent web applications**.
- It allows embedding **Java code** directly into **HTML pages**.
- JSP is built on top of **Java Servlets** and is part of the **Java EE** (Jakarta EE) specification.

Key Features

- **Dynamic Content:** Generates HTML, XML, or other types of documents dynamically.
- **Platform Independent:** Runs on any server supporting Java.
- **Reusable Components:** Can use JavaBeans, custom tags, and tag libraries.
- **Separation of Concerns:** HTML for presentation, Java for business logic.
- **Access to Java APIs:** JDBC, EJB, etc.

JSP Lifecycle

JSP pages are internally converted into **Servlets** by the server.

The lifecycle has **5 main phases**:

1. **Translation:** JSP → Servlet source code.
2. **Compilation:** Servlet source → Servlet class.
3. **Loading:** Servlet class is loaded into memory.
4. **Instantiation & Initialization:** `jspInit()` method is called once.
5. **Request Processing:** `_jspService()` method handles each request.
6. **Destruction:** `jspDestroy()` method is called before unloading.

Basic Syntax Elements

a) Directives

- Provide global information to the JSP container.
- Syntax: `<%@ directive attribute="value" %>`
- Types:
 - **page:** Defines page-level settings (e.g., `<%@ page language="java" contentType="text/html" %>`).

- **include:** Includes static resources (e.g., `<%@ include file="header.jsp"%>`).
- **taglib:** Declares custom tag libraries.

b) Scriptlets

- Embed Java code inside HTML.
- Syntax: `<% Java code %>`

c) Expressions

- Output values directly to the client.
- Syntax: `<%= expression %>`

d) Declarations

- Declare variables or methods.
- Syntax: `<%! declaration %>`

Example: Hello JSP

Jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<head><title>Hello JSP</title></head>
<body>
  <h1>Welcome to JSP!</h1>
  <p>Current Time: <%= new java.util.Date() %></p>
</body>
</html>
```

Advantages of JSP

- Easier to write and maintain than Servlets for UI-heavy pages.
- Automatic compilation and deployment by the server.
- Can use **MVC(Model View Controller)** architecture with Servlets as controllers and JSP as views.

❖ JSP vs. Servlet

While both are used for dynamic web content, they serve different primary roles in an application.

Feature	Servlet	JSP
Primary Language	Java-based (HTML is written inside Java)	HTML-based (Java is embedded in HTML)
Role in MVC	Acts as the Controller (handles logic)	Acts as the View (handles presentation)
Performance	Faster (directly executed as a class)	Slower (must be translated to a servlet first)
Maintenance	Challenging for UI changes	Easier for UI changes (separation of concerns)
Implicit Objects	No built-in implicit objects	Provides 9 built-in implicit objects

❖ JSP Architecture

JSP follows a three-tier architecture involving a Client, Web Server/JSP Engine, and Database.

1. Client Layer: The browser sends an HTTP request for a file.
2. Web Server Layer: The JSP engine (e.g., Apache Tomcat) intercepts the request. It checks if the JSP has been modified; if so, it translates the JSP into a Java Servlet, compiles it into a file, and executes it.
3. Database Layer: The generated servlet interacts with a backend database to retrieve or store data.

JSP Architecture

1. Introduction

- **JSP (JavaServer Pages)** is a server-side technology used to create **dynamic web content**.
- It is built on top of the **Servlet API** and follows the **Servlet-based architecture**.
- JSP pages are essentially **HTML pages with embedded Java code** that the server processes.

JSP Processing Flow

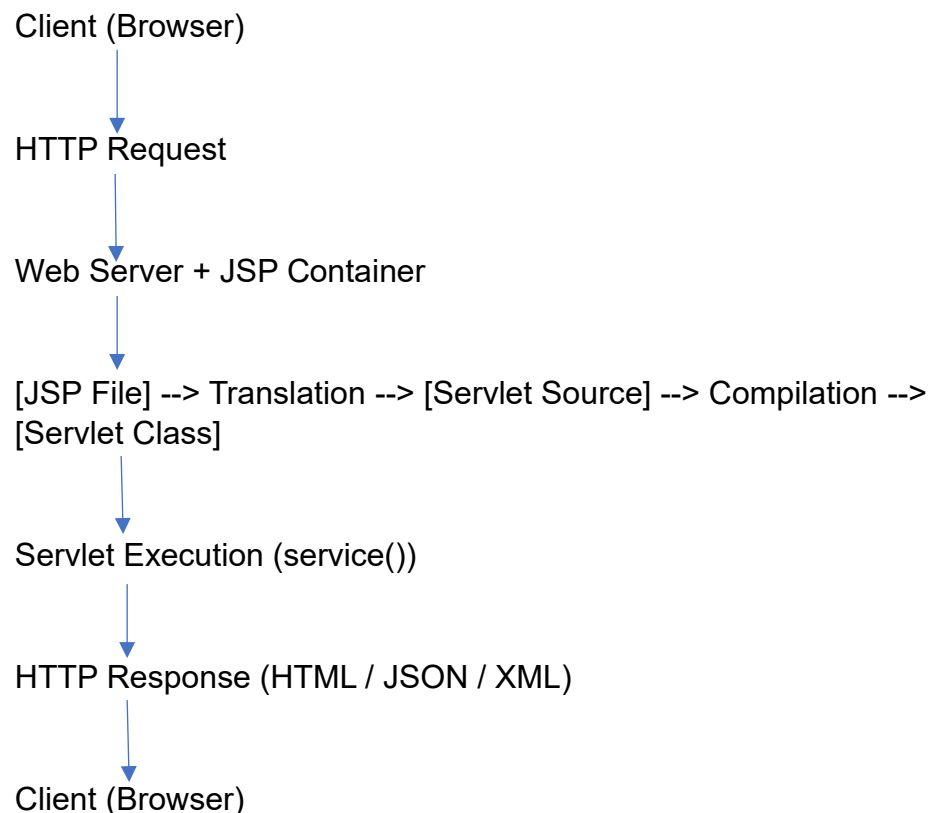
The JSP architecture works in **two main phases**:

- (a) **Translation & Compilation Phase** (first request)
- (b) **Execution Phase** (subsequent requests)

Step-by-Step Flow

1. **Client Request**
 - A web browser sends an HTTP request for a .jsp page to the web server.
2. **JSP Engine / Container**
 - The JSP engine (part of the web server, e.g., Apache Tomcat) receives the request.
3. **Translation**
 - The JSP file is **translated into a Java Servlet** source file by the JSP engine.
4. **Compilation**
 - The generated servlet source file is **compiled into a .class file** (bytecode).
5. **Class Loading & Instantiation**
 - The servlet class is loaded into the JVM and an instance is created.
6. **Execution**
 - The container calls the servlet's service() method, which executes the embedded Java code and generates dynamic content.
7. **Response to Client**
 - The generated HTML (or other content) is sent back to the client's browser.
8. **Subsequent Requests**
 - For later requests, the already compiled servlet is reused (unless the JSP file changes).

JSP Architecture Diagram



Key Components

- **JSP Page** – Contains HTML + Java code.
- **JSP Engine** – Translates JSP into a servlet.
- **Servlet Container** – Manages servlet lifecycle.
- **JVM** – Executes compiled servlet bytecode.
- **HTTP Protocol** – Communication between client and server.

Advantages of JSP Architecture

- **Separation of concerns** – HTML for presentation, Java for logic.
- **Automatic servlet generation** – No need to manually write servlets.
- **Reusability** – Can use JavaBeans, custom tags, and tag libraries.

❖ Life Cycle of JSP

A JSP page goes through several distinct phases from its first request to its destruction.

1. Translation: The JSP container parses the JSP file and translates it into a Java Servlet source file ().
2. Compilation: The container compiles the file into a Java Servlet class file ().
3. Class Loading: The container loads the compiled servlet class into memory.
4. Instantiation: The container creates an instance of the servlet class.
5. Initialization (): The method is called once to initialize resources like database connections.
6. Request Processing (): The method is invoked for every client request to generate the dynamic response. It cannot be overridden.
7. Destruction (): The method is called once before the JSP is removed from service, typically for cleanup tasks.

❖ JSP Elements

JSP elements allow you to insert Java logic into your static HTML content.

- **Scriptlets** : Used to write blocks of Java code that are placed inside the `<% %>` method.
- **Declarations** : Used to define variables and methods at the class level (outside the service method).
- **Expressions** : Used to output data directly to the client. The result is automatically converted to a string and printed.
- **Directives** : Used to provide global information for the entire page, such as importing classes (`<%@ %>`) or including files.
- **Comments** : Used to add comments that are ignored by the JSP engine and not sent to the client.

❖ JSP Elements

1. Directives Elements

Directives are instructions given to the JSP container (like Tomcat) when the page is being translated into a Servlet. They **do not produce any direct output** to the browser; instead, they configure how the entire page behaves.

- **Syntax:** `<%@ directive attribute="value" %>`

There are three main types of directives:

A. The page Directive

Defines page-dependent attributes, such as importing Java classes, defining the content type, or handling error pages.

- **Example:**

```
<%@ page language="java" contentType="text/html; charset=UTF-8" import="java.util.Date, java.util.ArrayList" %>
```

B. The include Directive

Used to merge the content of another file (HTML, JSP, or text) into the current JSP page **during the translation phase** (statically). Great for headers and footers!

- **Example:**

```
<%@ include file="header.jsp" %>
```

C. The taglib Directive

Declares a custom tag library (like JSTL) that the page can use, allowing you to replace messy Java code with clean, XML-like tags.

- **Example:**

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

2. Scripting Elements

Scripting elements allow you to insert Java code directly into your HTML.

Element Type	Syntax	Purpose & Behavior	Example
Declaration	<code><%! ... %></code>	Declares variables or methods at the class level. These become fields/methods of the generated Servlet.	<code><%! int globalCounter = 0; %></code>
Scriptlet	<code><% ... %></code>	Executes any valid Java code block . Code here goes inside the Servlet's service method.	<code><% if(user.isLoggedln()) { out.print("Welcome!"); } %></code>

Element Type	Syntax	Purpose & Behavior	Example
Expression	<code><%= ... %></code>	Evaluates an expression, converts it to a String, and prints it directly to the output. <i>Do not use a semicolon at the end!</i>	Current Time: <code><%= new java.util.Date() %></code>

3. Action Elements

Action elements are XML-like tags that control the behavior of the JSP engine dynamically at **request time** (runtime). They follow the XML syntax format.

Here are the 7 key action elements you need to know:

A. `jsp:useBean`

Locates or instantiates a Java Bean (a simple Java class with a no-arg constructor and getters/setters). This bridges the gap between your backend Java logic and frontend display.

```
<jsp:useBean id="user" class="com.classroom.User" scope="session" />
```

B. `jsp:setAttribute` & `jsp:getAttribute`

- `jsp:setAttribute`: Sets a value into a specific property of a Java Bean.
- `jsp:getAttribute`: Fetches a property value from a Java Bean and displays it on the screen.

```
<jsp:setAttribute name="user" property="username" value="Alex" />
```

```
Hello, <jsp:getAttribute name="user" property="username" />!
```

C. `jsp:include`

Includes the response of another resource (JSP, Servlet, or HTML) **at runtime**. Unlike the include directive (which copies-and-pastes code during translation), this one intercepts the request dynamically.

```
<jsp:include page="sidebar.jsp" />
```

D. `jsp:forward`

Forwards the current client request to a completely different JSP page or Servlet. The URL in the browser address bar stays the same, but the content changes.

```
<jsp:forward page="login_error.jsp" />
```

E. `jsp:param`

Used inside `<jsp:include>` or `<jsp:forward>` tags to pass parameters (key-value pairs) dynamically to the target page.

```
<jsp:forward page="dashboard.jsp">
```

```
    <jsp:param name="userRole" value="admin" />
```

```
</jsp:forward>
```

F. jsp:plugin

Instructs the web browser to download and execute a browser plugin (like a Java Applet or Flash component).

(note: This tag is mostly deprecated in modern web standards since Applets are dead, but it remains a classic part of the J2EE specification history).

```
<jsp:plugin type="applet" code="Clock.class" codebase="applets" />
```

❖ JSP Implicit Objects

Every implicit object is an instance of a specific Java class or interface.

Object Name	Associated Java Class / Interface	Main Scope	What it actually does
request	jakarta.servlet.http.HttpServletRequest	Request	Contains all data sent by the client (form data, cookies, headers).
response	jakarta.servlet.http.HttpServletResponse	Page	Used to configure the server's reply (redirects, content-type, cookies).
out	jakarta.servlet.jsp.JspWriter	Page	The output stream used to write HTML text directly to the browser.
session	jakarta.servlet.http.HttpSession	Session	Stores user-specific data across multiple page requests (e.g., shopping carts).
application	jakarta.servlet.ServletContext	Application	Stores global data shared across <i>all</i> users and <i>all</i> pages in the app.
pageContext	jakarta.servlet.jsp.PageContext	Page	The "master key" object. Provides access to all scopes and page details.

A. The `request` Object

Every time a user clicks a link or submits a form, a request is born. This object holds everything the client sent over.

Common Use Case: Reading HTML form inputs or URL parameters.

Code Example (Scriptlet):

```
<%  
    // Fetching a value from an input field named "username"  
    String username = request.getParameter("username");  
    out.print("Welcome back, " + username);  
%>
```

B. The `response` Object

This object represents the server's answer back to the client. You use it to manipulate the HTTP response header or send status codes.

Common Use Case: Redirecting a user to a login page if they aren't authorized.

Code Example (Scriptlet):

```
<%  
    // Automatically send the user to a different page  
    response.sendRedirect("login.jsp");  
%>
```

C. The `out` Object

This is your implicit writing pen. It is an asynchronous buffered output stream used to write data into the HTTP response body.

Common Use Case: Printing dynamic strings or loop elements into the HTML structure.

Code Example (Scriptlet):

```
<%  
    for(int i = 1; i <= 3; i++) {  
        out.print("<p>Line number " + i + "</p>");  
    }  
%>
```

D. The `session` Object

HTTP is stateless—it forgets who you are the millisecond a page loads. The `session` object fixes this by keeping track of a unique user as they browse your site.

Common Use Case: Keeping a user logged in, or tracking a shopping cart.

Code Example (Scriptlet):

```
<%  
    // Storing a value in the user's session  
    session.setAttribute("userRole", "admin");  
    // Retrieving it later  
    String role = (String) session.getAttribute("userRole");  
%>
```

E. The `application` Object

While `session` is personal to *one* user, `application` is global. It represents the entire web application running on the server. Data stored here is shared by every single user visiting your site.

Common Use Case: Creating a global website visitor counter.

Code Example (Scriptlet):

```
<%  
    // Global attribute shared by everyone  
    application.setAttribute("appName", "E-Learning Portal");  
%>
```

F. The `pageContext` Object

Think of `pageContext` as the manager of the current JSP page. It stores references to all other implicit objects and provides a single point of access to find attributes across different scopes.

Common Use Case: Explicitly searching for or setting variables in a specific scope (Page, Request, Session, or Application).

Code Example (Scriptlet):

```
<%  
    // Setting an attribute specifically in the Session scope using pageContext  
    pageContext.setAttribute("myVar", "SecretValue", PageContext.SESSION_SCOPE);  
%>
```

❖ JSP Scopes

What is a "Scope" in JSP?

In JSP, a scope defines the **visibility** and **lifespan** of an attribute (a Java object stored as a key-value pair).

All scopes provide two fundamental methods to manage data:

- `setAttribute("key", object)` — To store data.
- `getAttribute("key")` — To retrieve data.

The 4 JSP Scopes

A. Page Scope

This is the most restricted, localized scope. Data stored here is only accessible within the **exact JSP page** where it was created.

- **Lifespan:** Starts when the page request begins and ends as soon as the page finishes rendering and sends the HTML response back to the client.
- **Implicit Object Used:** `pageContext`
- **Real-world analogy:** A scratchpad on your desk. Once you finish that specific task and leave the desk, you throw the paper away.
- **Code Example:**

```
<%  
    pageContext.setAttribute("localMessage", "Only visible on this page!",  
    PageContext.PAGE_SCOPE);  
%>
```

B. Request Scope

Data stored here is accessible by any web component (JSP or Servlet) that handles the **same HTTP request cycle**.

- **Lifespan:** Lives as long as the request survives. If you use `<jsp:forward>` or a server-side `RequestDispatcher.forward()` to pass the user to another page, the data travels with them! However, if the user hits "Refresh" or clicks a new link, a brand-new request is born, and the old data is wiped out.
- **Implicit Object Used:** `request`
- **Real-world analogy:** A traveler's suitcase. As long as you stay on the same trip (even if you transfer flights via server forwarding), your luggage stays with you.
- **Code Example:**

```
<%  
    request.setAttribute("errorMessage", "Invalid password. Try again.");  
%>  
  
<jsp:forward page="login.jsp" />
```

C. Session Scope

Data stored here is tied to a **specific user** across multiple pages and multiple requests.

- **Lifespan:** Starts the moment a user visits your web app for the first time. It ends when the user closes their browser, logs out explicitly (`session.invalidate()`), or when the session times out due to inactivity (typically 30 minutes by default).
- **Implicit Object Used:** `session`
- **Real-world analogy:** A locker at a gym. It belongs exclusively to you for the duration of your visit. You can walk around, leave your locker, come back, and your stuff is still there.

- **Code Example:**

```
<%  
    // Storing a user object once they log in successfully  
    session.setAttribute("currentUser", loggedInUserObj);  
%>
```

D. Application Scope

This is the grandmaster scope. Data stored here is **global**—shared across every single user, every session, and every page in the entire web application.

- **Lifespan:** Starts when the web application is deployed/started on the server (e.g., Tomcat turns on) and dies only when the application is stopped or the server shuts down.
- **Implicit Object Used:** application
- **Real-world analogy:** A bulletin board in the gym lobby. Every single person who walks into the building can see it and read it.
- **Code Example:**

```
<%  
    // Accessing a shared database configuration string or counter  
    application.setAttribute("globalTitle", "Tech Academy Portal");  
%>
```

❖ **Including and Forwarding from JSP Pages**

- include Action
- forward Action

JSP Architecture: Including and Forwarding Requests

JSP provides two primary mechanisms to collaborate between web components: **Include** and **Forward**. These actions allow you to build modular, reusable, and maintainable web applications.

1. The <jsp:include> Action

The <jsp:include> action inserts the content of another resource (JSP, HTML, or Servlet) into the current JSP page **during the request-processing phase**.

Key Characteristics

- **Dynamic Inclusion:** The target page is executed at runtime, and its output is merged.
- **Control Flow:** Control transfers to the included resource and then **returns** to the main JSP.
- **Response Commitment:** The main page can write output before and after the include tag.

Syntax

```
<jsp:include page="relativeURL" flush="true" />
```

Passing Parameters

You can pass data to the included page using `<jsp:param>`:

```
<jsp:include page="sidebar.jsp">
    <jsp:param name="theme" value="dark" />
</jsp:include>
```

2. The `<jsp:forward>` Action

The `<jsp:forward>` action terminates the execution of the current JSP page and clears its buffer, shifting complete control to a new resource.

Key Characteristics

- **Control Flow:** Control transfers to the target resource and **does not return** to the calling JSP.
- **Single Response:** Only the target resource sends the final response to the client browser.
- **URL Behavior:** The browser's address bar does not change. The client thinks they are still on the original page.
- **Buffer Restriction:** You cannot write text to the client before the forward action unless the buffer is large enough not to flush.

Syntax

```
<jsp:forward page="relativeURL" />
```

Passing Parameters

```
<jsp:forward page="dashboard.jsp">
    <jsp:param name="userRole" value="admin" />
</jsp:forward>
```

Comparison: Include vs. Forward

Feature	<code><jsp:include></code>	<code><jsp:forward></code>
Control Flow	Goes to target, then returns to source.	Goes to target and stays there.
Output	Combines source and target outputs.	Displays target output only.
Common Use Case	Header, footer, sidebar navigation templates.	Authentication checks, MVC routing.
Execution Phase	Dynamic request-time inclusion.	Dynamic request-time redirection.

Code Examples

Scenario: A layout page including a header and forwarding users if not logged in.

index.jsp (Main Page)

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>
<html>
<head><title>Home Page</title></head>
<body>
    <!-- 1. Include the reusable header -->
    <jsp:include page="header.jsp" />
    <%
        // Simulate an authentication check
        String user = request.getParameter("username");
        if (user == null) {
    %>
        <!-- 2. Forward unauthorized users to login -->
        <jsp:forward page="login.jsp">
            <jsp:param name="error" value="Please log in first." />
        </jsp:forward>
    %>
        }
    %>
    <h1>Welcome to the Application Dashboard!</h1>
</body>
</html>
```

header.jsp (Included File)

```
<div style="background-color: #f1f1f1; padding: 10px; text-align: center;">
    <h2>My Application Website Header</h2>
</div>
```

❖ JSP Session Management: Cookies and Sessions

Web applications use the **HTTP protocol**, which is **stateless**. This means a server treats every request from a browser as completely new. To remember user data across multiple pages (like shopping carts or login states), JSP uses **Cookies** and **Sessions**.

1. Cookies in JSP

A cookie is a small text file stored by the web browser on the **client's local machine**.

Key Characteristics

- **Storage Location:** Client side (browser).
- **Security:** Low (users can view, modify, or disable cookies).
- **Data Type:** String text only.
- **Lifecycle:** Persists even after closing the browser if a maximum age is set.

Code Example: Creating and Reading Cookies

setCookie.jsp (Creating a Cookie)

```
<%
    // 1. Create a cookie object (Name, Value)
    Cookie userCookie = new Cookie("username", "Alex_Java");
    // 2. Set expiration time in seconds (24 hours)
    userCookie.setMaxAge(60 * 60 * 24);
    // 3. Send the cookie to the browser response
    response.addCookie(userCookie);
%>
<html>
<body>
    <h3>Cookie has been set!</h3>
    <a href="getCookie.jsp">Read the Cookie</a>
</body>
</html>
```

getCookie.jsp (Reading the Cookie)

```
<html>
<body>
<h3>Stored Cookies:</h3>
<%
    // 1. Retrieve all cookies sent by the browser
    Cookie[] cookies = request.getCookies();
    if (cookies != null) {
        for (Cookie c : cookies) {
            // 2. Check for the specific cookie name
```

```

        if (c.getName().equals("username")) {
            out.println("<p>Welcome back, " + c.getValue() + "!</p>");
        }
    }
} else {
    out.println("<p>No cookies found.</p>");
}
%>
</body>
</html>

```

2. HttpSession in JSP

A session is an object created on the **server side** to store unique user data across multiple page requests.

Key Characteristics

- **Storage Location:** Server side (memory).
- **Security:** High (the client only receives an encrypted session ID token).
- **Data Type:** Any Java Object (String, Lists, custom classes).
- **Lifecycle:** Expires automatically after a period of inactivity or when the browser closes.

Code Example: Managing User Sessions

startSession.jsp (Saving Session Data)

```

<%
    // 1. Access the implicit 'session' object and store data
    session.setAttribute("userRole", "Admin");
    session.setAttribute("cartCount", 5);
%>
<html>
<body>
    <h3>Session initialized and data stored!</h3>
    <a href="viewSession.jsp">View Session Data</a>
</body>
</html>

```

viewSession.jsp (Reading Session Data)

```
<html>

<body>

<h3>Session Details:</h3>

<%

    // 1. Retrieve data using the attribute name (Requires casting)
    String role = (String) session.getAttribute("userRole");
    Integer items = (Integer) session.getAttribute("cartCount");

    if (role != null) {
        out.println("<p>Role: " + role + "</p>");
        out.println("<p>Items in Cart: " + items + "</p>");
    } else {
        out.println("<p>No active session found.</p>");
    }
%>

<br>

<a href="logout.jsp">Log Out</a>

</body>

</html>
```

logout.jsp (Destroying the Session)

```
<%

    // 1. Invalidate and destroy the current session
    session.invalidate();
%>

<html>

<body>

    <h3>Logged out successfully. Session destroyed!</h3>

    <a href="viewSession.jsp">Try viewing session again</a>

</body>

</html>
```

3. Comparison: Cookie vs. Session

Feature	Cookies	Sessions
Storage	Client machine.	Server memory.
Capacity	Limit of 4KB per cookie.	Unlimited (depends on server memory).
Security	Vulnerable to tampering.	Secure from direct client tampering.
Performance	Slows requests if large (sent with every HTTP header).	Consumes server RAM.

❖ JSP Error and Exception Handling

Web applications encounter errors during execution due to bugs, invalid user inputs, or database failures. JSP offers two primary methods to handle these issues gracefully: **Page-Level Handlers** and **Application-Level Handlers**.

1. Page-Level Exception Handling

This approach targets individual JSP pages using the page directive attributes: `errorPage` and `isErrorPage`.

The Core Components

- **errorPage**: Specified in the working JSP page to point to an error-handling file if an unhandled exception occurs.
- **isErrorPage**: Specified in the target error page to unlock the implicit exception object.

Code Example

calculate.jsp (The Risky Page)

This page attempts a division by zero to intentionally trigger an `ArithmeticException`.

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
errorPage="errorHandler.jsp" %>
```

```
<html>
```

```
<head><title>Calculator Page</title></head>
```

```
<body>
```

```
<%
```

```
    // Intentionally generating an ArithmeticException
```

```
    int num1 = 10;
```

```
    int num2 = 0;
```

```
    int result = num1 / num2;
```

```

        out.println("Result is: " + result);
    %>
</body>
</html>

```

errorHandler.jsp (The Dedicated Error Page)

This page catches the error and displays a user-friendly message.

```

<%@ page language="java" contentType="text/html; charset=UTF-8" isErrorPage="true" %>
<html>
<head><title>Error Occurred</title></head>
<body style="background-color: #ffe6e6; font-family: Arial, sans-serif;">
    <h2>Oops! An error occurred on our end.</h2>
    <p><strong>Error Type:</strong> <%= exception.getClass().getName() %></p>
    <p><strong>Reason:</strong> <%= exception.getMessage() %></p>
    <!-- Optional: For system administrators to trace the issue -->
    <!-- <% exception.printStackTrace(new java.io.PrintWriter(out)); %> -->
    <br>
    <a href="calculate.jsp">Try Again</a>
</body>
</html>

```

2. Application-Level Exception Handling

Instead of writing `errorPage="url"` on every single JSP page, you can define global error pages inside the application configuration file (**web.xml**).

Configuration Options

You can configure global errors by **HTTP Status Code** (e.g., 404 Not Found) or by **Java Exception Type** (e.g., `NullPointerException`).

web.xml Configuration snippet

```

<web-app xmlns="https://jakarta.ee" version="6.0">
    <!-- Handle 404 Not Found globally -->
    <error-page>
        <error-code>404</error-code>
        <location>/missingPageError.jsp</location>
    </error-page>
    <!-- Handle explicit Java exceptions globally -->
    <error-page>

```

```

        <exception-type>java.lang.NullPointerException</exception-type>

        <location>/nullError.jsp</location>

    </error-page>

</web-app>

```

❖ JSP EL (Expression Language), JSP Standard Tag Libraries (JSTL)

1. JSP Expression Language (EL)

- **Purpose:** Simplifies access to data stored in JavaBeans, request parameters, session attributes, and application attributes.
- **Syntax:** `${expression}`
- **Features:**
 - Access scoped variables: `${requestScope.name}`, `${sessionScope.user}`
 - Access JavaBeans properties: `${student.name}`
 - Access collections: `${list[0]}`, `${map.key}`
 - Operators: `+`, `-`, `*`, `/`, `==`, `!=`, `&&`, `||`
 - Implicit objects:
 - `pageScope`, `requestScope`, `sessionScope`, `applicationScope`
 - `param`, `paramValues`, `header`, `cookie`, `initParam`

Example: EL Usage

```

jsp
<%@ page contentType="text/html" %>
<html>
<body>
    <h2>Welcome, ${sessionScope.username}!</h2>
    <p>Your email is: ${param.email}</p>
    <p>First item in list: ${myList[0]}</p>
</body>
</html>

```

2. JSP Standard Tag Library (JSTL)

- **Definition:** A collection of useful JSP tags that encapsulate core functionality.
- **Advantages:**
 - Reduces Java code in JSP pages.
 - Provides clean, maintainable JSPs.
 - Supports internationalization, SQL, XML, and core operations.

JSTL Tag Libraries

Library	URI	Purpose
Core	http://java.sun.com/jsp/jstl/core	Flow control, iteration, variable support
Formatting (fmt)	http://java.sun.com/jsp/jstl/fmt	Internationalization, number/date formatting
SQL	http://java.sun.com/jsp/jstl/sql	Database access (not recommended for production)
XML	http://java.sun.com/jsp/jstl/xml	XML data processing

3. JSTL Core Tags

- `<c:out>` → Outputs value of an expression.
- `<c:set>` → Sets a variable.
- `<c:if>` → Conditional execution.
- `<c:choose>`, `<c:when>`, `<c:otherwise>` → Switch-like logic.
- `<c:forEach>` → Iteration over collections.

Example: JSTL Core

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
<body>
  <c:set var="name" value="Jadeja" />
  <p>Hello, <c:out value="${name}" /></p>

  <c:if test="${age >= 18}">
    <p>You are eligible to vote.</p>
  </c:if>

  <c:forEach var="item" items="${myList}">
    <p>Item: ${item}</p>
  </c:forEach>
</body>
</html>
```

4. JSTL Formatting Example

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/fmt" prefix="fmt" %>
<html>
<body>
  <fmt:formatNumber value="12345.678" type="currency" />
  <br/>
  <fmt:formatDate value="${today}" pattern="dd-MM-yyyy" />
</body>
</html>
```